

Современные оптимизирующие компиляторы с организацией параллельной обработки

Оптимизация

Оптимизацией называется обработка, связанная с переупорядочиванием и изменением операций в компилируемой программе в целях получения более эффективного объектного кода.

Цель оптимизации

Основная цель оптимизации -
преобразование промежуточного
представления программы в целях
повышения эффективности результирующей
объектной программы.

Критерии оптимизации:

- минимальные затраты времени ЦП
- минимальные затраты оперативной памяти
- получение требуемой точности результатов
- получение нужной надежности работы информационной и управляющей системы

Определение оптимизирующего преобразования

- Преобразованием программы является любая функция p , определённым образом изменяющая внутреннее представление программы.
- Существует общий класс конкретизирующих преобразований (обобщающих, сужающих и эквивалентных), использующийся как при доказательстве правильности программ, так и при оптимизирующих трансформациях.

- Подкласс конкретизирующих преобразований - класс *анализирующих* преобразований, направленных на решение задач верификации и на решение задач потокового анализа при оптимизации программ.

Оптимизирующее преобразование в компиляторе должно:

- определить часть программы, которую надо оптимизировать, и определить соответствующее оптимизирующее преобразование;
- проверить, что преобразование не изменяет результат исполнения данного участка кода или изменяет его в рамках, утверждённых пользователем;
- провести преобразование.

Правило :

- Преобразование программы корректно, если для всех семантически правильных исполнений программы, оригинальная и преобразованная версии программы дают идентичный результат для идентичных запусков.

Участки экономии.

- Оптимизирующие преобразования обычно производятся над некоторым участком программы

Основные градации величины участков экономии :

- операторы – в основном арифметические операторы являются участком экономии в рамках одного оператора;
- базовый блок – последовательность операторов с единственной точкой входа и без ветвлений, базовый блок (ББ) был объектом многих ранних исследований по оптимизации;
- самый **внутренний** цикл - в этом контексте выполняются многие оптимизации;

Основные градации величины участков экономии :

- идеально вложенное гнездо циклов (все циклы, кроме самого вложенного содержат в себе только один оператор – более глубоко вложенный цикл);
- вложенное гнездо циклов (любого формата);
- процедура (глобальная оптимизация);
- множество процедур, рассматриваемых вместе (межпроцедурная оптимизация).

Основные градации величины участков экономии :

- Оптимизирующее преобразование применяется с целью улучшения некоторой характеристики (характеристик) программы.
- Зачастую преобразование при улучшении одной из характеристик, ухудшает другую.

Примеры характеристик

- количество занимаемых ресурсов процессора (например, функциональных устройств);
- минимизация количества выполняемых операций;
- минимизация количества обращений в оперативную память;
- минимизация размера программы и использования памяти для данных;
- минимизация энергопотребления.

Примеры характеристик

- Оптимизация программного кода — это модификация программ, выполняемая оптимизирующим **КОМПИЛЯТОРОМ**. Оптимизация — не обязательный, но важный этап компиляции. Она может происходить неявно во время трансляции программы
- Оптимизацию программы выделяют как отдельный этап функционирования компиляторов.
- Компоновщики так же могут выполнять часть оптимизаций, таких как удаление неиспользуемых подпрограмм или переупорядочивание подпрограмм.

Алгоритм оптимизации

1. Программа, поступающая на вход оптимизирующего компилятора, предварительно преобразуется в вид, удобный для анализа и оптимизаций.
2. Далее строятся специальные вспомогательные структуры данных
3. Избавление от лишних вычислений и нахождение наиболее часто выполняемой последовательности команд
4. Построение оптимального кода, включая планирование для архитектур с параллельными исполняющими устройствами.



Виды оптимизации

- Низкоуровневая оптимизация преобразовывает программу на уровне элементарных команд
- Высокоуровневая оптимизация осуществляется на уровне структурных элементов программы, таких как ветвления и циклы.

Низкоуровневая оптимизация

- Оптимальный выбор (замена, объединение, разделение) инструкций
- Переупорядочивание инструкций
- Распределение регистров
- Удаление цепочек переходов
- Векторизация
- Понижение силы операций

Высокоуровневая оптимизация

- Удаление недостижимого («мёртвого») кода и неиспользуемых присвоений
- Подгонка, обращение циклов
- Оптимизация множественных ветвлений
- Развёртка, свёртка, объединение и разделение циклов

Высокоуровневая оптимизация

- Вычисление инвариантов циклов, вынесение общих подвыражений и кода в ветвлениях, вынесение ветвлений из циклов
- Переключение, объединение и разделение ветвлений
- Предвыборка данных
- Переупорядочивание функций
- Встраивание и извлечение функций

Среди машинно-независимых методов можно выделить самые основные:

- Свертка, т.е. выполнение операций, операнды которых известны во время компиляции.
- Исключение лишних операций за счет однократного программирования общих подвыражений.
- Вынесение из цикла операций, операнды которых не изменяются внутри цикла.

Оптимизация внутри линейных участков

Линейным участком является выполняемая по порядку последовательность операций с одним входом и одним выходом (первая и последняя операции соответственно).

Представление линейного участка в виде триад

Линейный
участок:

В виде триад:

$$I := 1 + 1;$$

$$(1) + 1, 1$$

$$(2) := I, (1)$$

$$I := 3;$$

$$(3) := I, 3$$

$$B := 7 + I;$$

$$(4) + 7, (3)$$

$$(5) := B, (4)$$

Свертка

Свертка — это выполнение во время компиляции операций исходной программы, для которых значения операндов уже известны, и, поэтому, нет нужды их выполнять во время счета.

Свертка

Видно, что первую триаду можно вычислить во время компиляции и заменить результирующей константой. Менее очевидно, что четвертую триаду также можно вычислить, т.к. к моменту ее обработки известно, что I равно 3.

До свертки:

(1) + 1,1

(2) := I,(1)

(3) := I,3

(4) + 7,(3)

(5) := B,(4)

После свертки:

(1) := I,2

(2) := I,3

(3) := B,10

Свертка

Главным образом свертка применяется к :

- Арифметическим операциям, так как они наиболее часто встречаются в исходной программе.
- Операторам преобразования типа, причем проблема упрощается, если эти преобразования заданы явно, а не подразумеваются по умолчанию.

Свертка

Процесс свертки операторов, имеющих в качестве операндов константы, понятен и сводится к внутреннему их вычислению.

Свертка операторов, значения которых могут быть определены в результате некоторого анализа, несколько сложнее. Обычно свертку осуществляют только в пределах линейного участка при помощи таблицы.

Свертка

	Шаг 1	Шаг 2	Шаг 3	Шаг 4	Шаг 5
1	+ 1,1	=2	C 2	C 2	C 2
2	:= I,(1)	:= I,(1)	:= I,2	:= I,2	:= I,2
3	:= I,3	:= I,3	:= I,3	:= I,3	:= I,3
4	+ 7,(3)	+ 7,(3)	+ 7,(3)	C 10	C 10
5	:= B,(4)	:= B,(4)	:= B,(4)	:= B,(4)	:= B,10
T			(I,2)	(I,3)	(I,3)
T					(B,10)

Завершение алгоритма свертки

Свертка

С точки зрения работы компилятора процесс свертки является дополнительным проходом по внутреннему представлению исходной программы представленной триадами. Но свертку можно проводить и с тетрадами. Кроме того, можно оптимизировать программу в семантических программах во время получения внутреннего представления и при этом отпадает необходимость в дополнительном проходе

Исключение лишних операций

i -я операция линейного участка считается лишней, если существует более ранняя идентичная j -я операция и никакая переменная от которой зависит эта операция, не изменяется третьей операцией лежащей между i -й и j -й операциями

Исключение лишних операций

Например на линейном участке

$$D:=D+C*B$$

...

$$A:=D+C*B$$

можно выделить лишние операции. Если представить линейный участок в виде триад

Исключение лишних операций

(1) * C,V

(2) + D,(1)

(3) := D,(2)

(4) * C,V

(5) + D,(4)

(6) := A,(5)

операция $C*V$ во второй раз лишняя, так как, ни C ни V не изменяются после триады (1)

Исключение лишних операций

Алгоритм исключения лишних операций просматривает операции в порядке их появления. И если i -я триада лишняя, так как уже имеется идентичная ей j -я триада, то она заменяется триадой $(SAME, j, 0)$, где операция $SAME$ ничего не делает и не порождает никаких команд при генерации

Исключение лишних операций

Для слежения за внутренней зависимостью переменных и триад, можно поставить им в соответствие числа зависимостей (dependency numbers).

При этом используются следующие правила:

- Вначале для переменной A число зависимости $\text{dep}(A)$ выбирается равным нулю, так как ее значение не зависит ни от одной триады

Исключение лишних операций

- После обработки i -й триады, где переменной A присваивается какое-либо значение число зависимости становится равным i , так как ее новое значение зависит от i -й триады;
- При обработке i -й триады ее число зависимостей $\text{dep}(i)$ становится равным максимальному из чисел зависимостей ее операндов $+ 1$.

Исключение лишних операций

Числа зависимостей используются следующим образом: если i -я триада идентична j -й триаде ($j < i$), то i -я триада является лишней тогда и только тогда, когда $\text{dep}(i) = \text{dep}(j)$.

Исключение лишних операций

Обрабатываемая триада I	Dep (переменная) A B C D	Dep(i)	Полученная в результате триада
(1) * C,B	max 0(0 0)0 +1=	1	(1) * C,B
(2) + D,(1)	0 0 0 0	1	(2) + D,(1)
(3) :=D,(2)	0 0 0 0	1	(3) :=D,(2) Dep(D)=3
(4) * C,B	max 0(0 0)3 +1=	1	(4) SAME 1
(5) + D,(4)	0 0 0 3	4	(5) + D,(4)
(6) :=A,(5)	0 0 0 3	1	(6) :=A,(5) Dep(A)=6
	6 0 0 3		

Обнаружена операция присваивания

Изменение числа зависимости переменной A

Вынесение инвариантных операций за тело цикла

Инвариантными называются такие операции, при которых ни один из операндов не изменяется внутри цикла.

В общем случае данная оптимизация выполняется двойным просмотром тела цикла с анализом операндов каждой из операций внутри цикла. В случае если они инвариантны, операция выносится назад за тело цикла, а внутри цикла выбрасывается.

Вынесение инвариантных операций за тело цикла

Для проверки инвариантности переменных используют специальную таблицу инвариантности.

Во время первого прохода цикла заполняется таблица инвариантных переменных, во время второго — выполняется собственно вынесение операций.

Примеры оптимизации

Продвижение констант

- **Цель оптимизации:** Уменьшение избыточных вычислений.
- **Пути достижения:** Продвижение константных значений вниз по коду. Обращение к переменной заменяется её константным значением.

Оригинальный код	После продвижения констант
<pre>int i,n,c; int a[64]; n = 64; c = 3; for(i = 0; i < n; i++) a[i] = a[i] + c;</pre>	<pre>int i; int a[64]; for(i = 0; i < 64; i++) a[i] = a[i] + 3;</pre>

Распространение копий

- **Цель оптимизации:** Уменьшение числа избыточных переменных содержащих одно и то же значение.
- **Пути достижения:** Использование оригинала вместо копий значения.

Оригинальный код	После распространения копий
<pre>int a[15]; int t = i * 4; int s = t; int c = 3; cout << a[s]; int r; r = t; a[r] = a[r] + c;</pre>	<pre>int a[15]; int t = i*4; int c = 3; cout << a[t]; a[t] = a[t] + c;</pre>

Подстановка операторов

- **Цель оптимизации:** Изменение зависимостей между переменными или улучшение возможностей анализа индексов в циклах.
- **Пути достижения:** Использование переменной заменяется выражением.

Оригинальный код	После подстановки операторов
<pre>int np1 = n + 1; for(i = 0; i < n; i++) a[np1] = a[np1] + a[i];</pre>	<pre>for(i = 0; i < n; i++) a[n+1] = a[n+1] + a[i];</pre>

Прямое преобразование

- **Цель оптимизации:** Снижение стоимости выполнения операций.
- **Пути достижения:** Замена операций эквивалентными с меньшей стоимостью исполнения.

Оригинальный код	После прямого преобразования
<pre>int x; double y; y = y * 2; x = x * 4; x = x / 2;</pre>	<pre>int x, y; y = y + y; x = x << 2; x = x >> 1;</pre>

Удаление неиспользуемого кода

- **Цель оптимизации:** Сокращение объема программы за счет удаления неиспользуемого кода.
- **Пути достижения:** Неиспользуемые блоки могут быть обнаружены после использования продвижения констант, анализа неиспользуемых констант.

Оригинальный код

```
int x = 0;  
if( x > 1 ) {  
    function1();  
}  
while( x > 0 ) {  
    function2();  
}  
function3();
```

После продвижения констант, свертки константных выражений и удаления неиспользуемого кода

```
function3();
```

Упрощение булевых выражений в серию переходов

- Цель оптимизации: Сокращение вычислений.
Пути достижения: Значение некоторых булевых выражений может быть определено по первому операнду.

Оригинальный код	После упрощения
<pre>int x,y,c; if(x == 1 && y == 2) c = 5;</pre>	<pre>int x,y,c; if(x == 1) if(y == 2) c = 5;</pre>

Раскрутка циклов

- **Цель оптимизации:** Увеличение параллелизма инструкций (увеличение количества инструкций, которые потенциально могут выполняться параллельно), “улучшение” использования регистров и кэша данных.
- **Пути достижения:** Повторение тела цикла несколько раз.

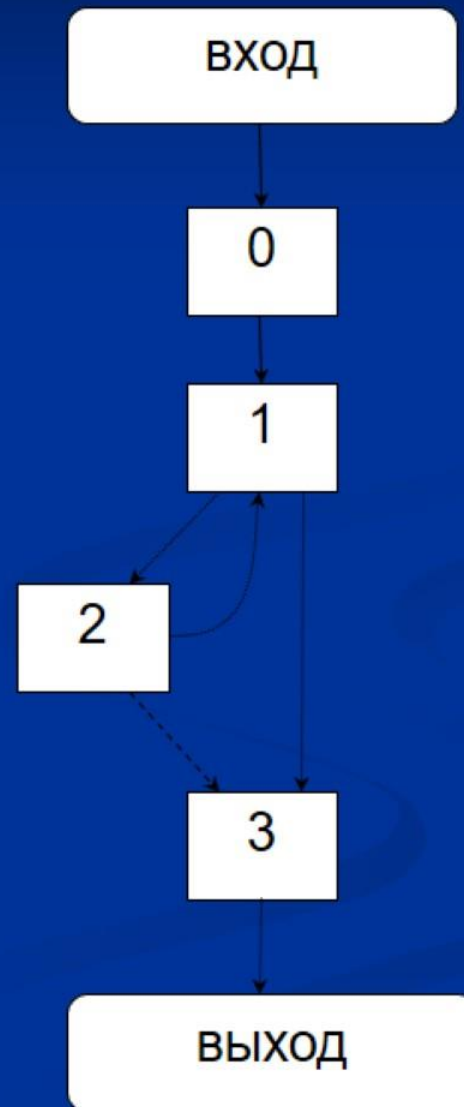
Оригинальный код	После раскрутки цикла
<pre>for (i = 1; i < n-1; i++) a[i] = (a[i-1] + a[i+1]) / 2;</pre>	<pre>for (i = 1; i < n-2; i+=2) { a[i] = (a[i-1] + a[i+1]) / 2; a[i+1] = (a[i] + a[i+2]) / 2; ; } if ((n-2) % 2 == 1) a[n-2] = (a[n-3] + a[n-1]) / 2;</pre>

Вынесение условных выражений за пределы цикла

- **Цель оптимизации:** Уменьшение накладных расходов на проверку условия с инвариантной относительно цикла переменной.
- **Пути достижения:** Перенос цикла в ветви условного оператора

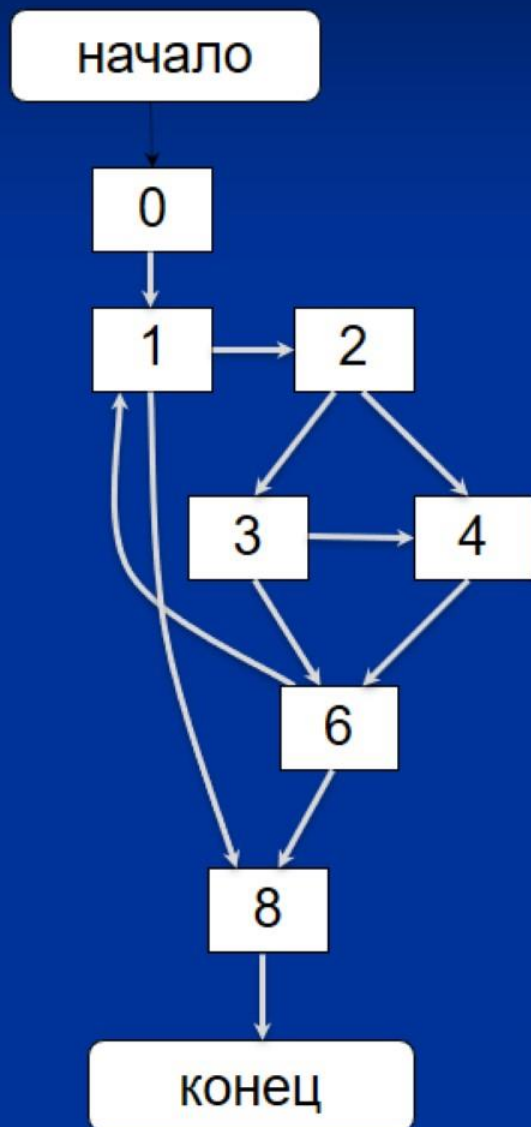
Граф потока управления для цикла

```
void foo (void) {  
0: int i;  
0: i = 0;  
1: while (i < n) {  
2:     /*тело цикла*/  
3: }  
}
```



Исходный код	Код после вынесения условного выражения
<pre>int n = 64; int a [64]; int b [64]; void foo (void) { 0:int i = 0; 0:int c; 1:while (i < n){ 2: a[i]=a[i]+4; 2: if (c<10) 3: b[i]=a[i]*b[i-1]; 2: else 4: b[i]=a[i]+6; 6: i++; 8:} }</pre>	<pre>int n = 64; int a [64]; int b [64]; void foo (void) { 0:int i = 0; 0:int c; 0:if (c < 10) 2: while (i < n) { 3: a [i] = a[i]+4; 3: b[i]=a[i]*b[i-1]; 3: i++; } 0:else 7: while (i < n) { 8: a [i] = a[i]+4; 8: b[i]=a[i]+6; 8: i++; 9: } }</pre>

**Исходный граф потока
управления**



**Граф потока управления после
вынесения условного выражения**



Вынесение первых и последних итераций

- **Цель оптимизации:** Удаление зависимостей, вызванных несколькими первыми или последними итерациями.
- **Пути достижения:** Вынесение нескольких первых или последних итераций за пределы цикла

Исходный текст	Код после вынесения первой итерации второго цикла
<pre> int n = 64; int a [64]; int b [64]; void foo (void) { 0: int i = 3; 1: while (i < n) { 2: b [i] = b [i] + b [2]; 2: i++; 4: } i = 2; 5: while (i < n) { 6: a [i] = a [i] + 3; 6: i++; 9: } } </pre>	<pre> int n = 64; int a [64]; int b [64]; void foo (void) { 0: int i = 2; 0: if (i <= 2) 1: a [i] = a [i] + 3; 2: i = 3; 3: while (i < n) { 4: a [i] = a [i] + 3; 4: b [i] = b [i] + b[2] 4: i++; 7: } } </pre>

Оптимизация хвостовых вызовов

- **Цель оптимизации:** Уменьшить накладные расходы при вызове функций.
- **Пути достижения:** Замена вызова (call) на безусловный переход (jmp).

Исходная программа	
<pre>int bar (int a) { printf ("bar!"); if (a == 0) return -1; return a; }</pre>	<pre>int foo (int a) { printf ("foo!"); if (a == 0) return 0; return bar (a); }</pre>

Оптимизация вызовов отключена

```
bar:
pushl %ebp
movl   %esp, %ebp
...
leave
ret
foo:
pushl %ebp
movl   %esp, %ebp
...
subl   $12, %esp
pushl  8(%ebp)
call   bar
addl   $16, %esp
...
leave
ret
```

Оптимизация вызовов включена

```
bar:
pushl %ebp
movl   %esp, %ebp
...
leave
ret
foo:
pushl %ebp
movl   %esp, %ebp
...
movl   8(%ebp), %eax
movl   %eax, 8(%ebp)
leave
jmp     bar
...
leave
ret
```

Встраивание функций (Inline)

- **Цель оптимизации:** Снижение накладных расходов на вызовы функций (за счет увеличения размера).
- **Пути достижения:** Копирование тела встраиваемой функции в вызывающую функцию. Конкретная реализация определяется компилятором. Обычно есть ряд ограничений на разрастание кода и на присутствие в коде различных управляющих структур, например вызова других функций.

Оригинальный код

```
void bar (int* a, int  
    i) {  
a[i]=a[i]+3;  
}  
void foo (void) {  
int i;  
for (i = 3; i < n;  
    i++)  
bar (a, i);  
}
```

После встраивания

```
void foo (void) {  
int i;  
for (i = 3; i < n;  
    i++)  
a[i]=a[i]+3;  
}
```

Оптимизирующие компиляторы

Оптимизирующие компиляторы

- Оптимизирующие компиляторы, используя различные оптимизации, способны существенно распараллеливать исходную задачу на уровне элементарных операций.
- Оптимальное планирование параллельного потока операций является одной из важнейших задач оптимизирующей компиляции

Оптимизирующие компиляторы

- В современных оптимизирующих компиляторах, имеющих в своём распоряжении большое количество оптимизирующих преобразований, на первый план выходят проблемы **структурирования оптимизаций и организации их взаимодействия.**

Оптимизирующие компиляторы

- Для эффективного использования своих потенциальных возможностей компилятору необходимо правильно подобрать последовательность запусков оптимизаций
- Важно для промышленного компилятора, имея внушительный список оптимизаций, уложиться в допустимое время при компиляции программы.

Примеры современных компиляторов типа Fortran

Intel Visual Fortran Compiler

Версия программы: 11.0.072

Professional Edition

- Снабжен полноценной поддержкой технологии 64-битной адресации памяти Intel Extended Memory 64 Technology (Intel EM64T) и технологии многопоточной обработки OpenMP 3.0
- способен автоматически распараллеливать процессы.
- позволяет достичь уменьшения объема и повышение производительности исполняемого кода приложений на 32- и 64-разрядных платформах Intel, включая системы на базе новейших процессоров Pentium M
- предлагает наилучшую поддержку для создания многопоточных приложений.

Intel Visual Fortran Compiler

- Разработчикам теперь предоставляется три варианта выбора компилятора в зависимости от требуемой математической обработки:
 - Комплект Professional Edition объединяет в себе высокую производительность с помощью библиотеки Intel® Math Kernel Library (Intel® MKL).
 - Версия Professional Edition с библиотекой IMSL* также включает библиотеку IMSL Fortran Library для ОС Windows.
 - У компилятора выпуска Standard Edition те же самые характеристики и производительность как и у компилятора выпуска Professional Edition, однако, он не содержит библиотеку производительности Intel MKL или математическую библиотеку IMSL.
- Все выпуски теперь включают в себя инструментальные средства Microsoft Visual Studio 2005 Premier Partner Edition, обеспечивая полную среду программной разработки Фортран.

Intel Visual Fortran Compiler

■ Характеристики программы:

- показывает стремительное развитие и рекордную производительность для всего диапазона платформ на основе процессоров компании Intel.
- Автоматически оптимизируется и распараллеливается программное обеспечение для того, чтобы наилучшим образом использовать функции многоядерных процессоров компании Intel
- Предоставляет инструментальные программные средства многоядерных процессоров, что достигается объединением компилятора Фортран и встроенных в него функций оптимизации, организации потоков и надежности с высоко оптимизированной математической библиотекой

Intel Visual Fortran Compiler

- Поддержка многопоточковых приложений, включая систему OpenMP и автоматическую параллелизацию для организации простых и эффективных потоков данных программного обеспечения.
- Параллельный высокопроизводительный оптимизатор (HPO) преобразует и оптимизирует циклы, чтобы автоматическая векторизация, система OpenMP
- Межпроцедурная оптимизация (IPO) эффективно улучшает производительность часто используемых функций малого и среднего размера, особенно программ, содержащих вызовы внутри циклов.
- Оптимизация на основе профиля (PGO) улучшает производительность приложений, уменьшая пробуксовку выполнения команд работы с кэш-памятью, изменяя компоновку кода, сжимая его размер и уменьшая вероятность ошибочного прогнозирования ветви исполняемой параллельной программы.

Нововведения:

- Поддержка OpenMP* 3.0
- Поддержка инструкций Intel(R) SSE4 для процессоров Intel(R) Core(TM) i7
- Улучшена производительность
- Родная поддержка x64 (сам компилятор на системах x64 устанавливается в варианте x64)
MKL 10.1

Компилятор Intel® Fortran Compiler Professional Edition для ОС Linux

■ Характеристики:

- Совместимость с инструментами GNU
- Поддержка многопоточных приложений, в том числе новые функции версии 11.0, OpenMP 3.0
- Высокопроизводительное средство оптимизации параллельной обработки (HPO)
- Межпроцедурная оптимизация (IPO) значительно повышает производительность функций малого и среднего размера, и особенно программ, содержащих вызовы внутри циклов.
- Отладка оптимизированного кода с помощью Intel® Debugger для приложений на базе архитектуры IA-32 и Intel® 64 повышает эффективность процесса отладки кода, оптимизированного для архитектуры Intel®

Новое в данной версии Intel® Fortran Compiler

- Дополнительная поддержка Fortran 2003
- OpenMP отделяет абстракцию параллелизма от прикладных интерфейсов, упрощая многопоточную оптимизацию и перенос кода
- Поддержка SSE2 включена по умолчанию
- Поддерживает сборку посредством распределения файлов между процессорами, оптимизируя код для использования с многоядерными процессорами и ускоряя цикл редактирования, компиляции и отладки
- Более подробная диагностика оптимизации
- Поиск и анализ проблем с исходными файлами

Новое в данной версии Intel® Fortran Compiler

- Графический интерфейс на базе клиента Eclipse наглядно представляет параллелизм приложений. Также обеспечена поддержка интерфейса командной строки
- Новая архитектура обеспечивает максимальную поддержку различных конфигураций разработки и разнообразных процессоров в одном корпусе.
- Новый уровень многопоточности
- Функция DftiCopyDescriptor добавлена для удобства при использовании быстрых преобразований Фурье
- Поддержка новых ОС Linux Fedora 9, Ubuntu 8.04, средства GNU 4.2 и 4.3.